

198. House Robber

Solved 

Medium

Topics

Companies

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1, 2, 3, 1]`

Output: 4

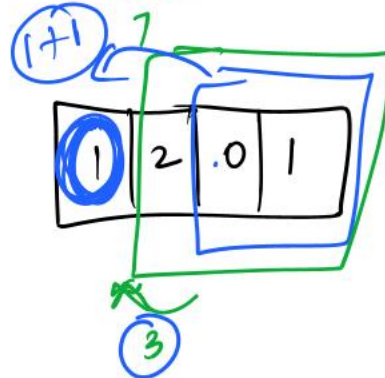
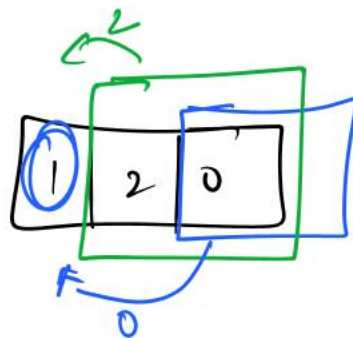
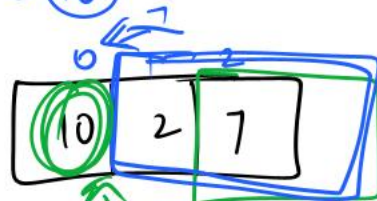
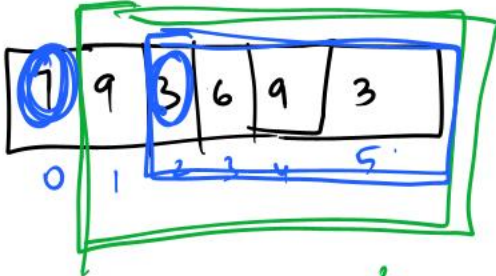
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

Robbing (0, 2) $\Rightarrow$ 4 $1 + 3 = 4$ 

$$2 + 9 + 1 \Rightarrow 12$$

$$7 + 3 \Rightarrow 10$$



Rob the current house and skip the next.

skip the current house and rob the next one.

... (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

#



1 3 5 . . .

14

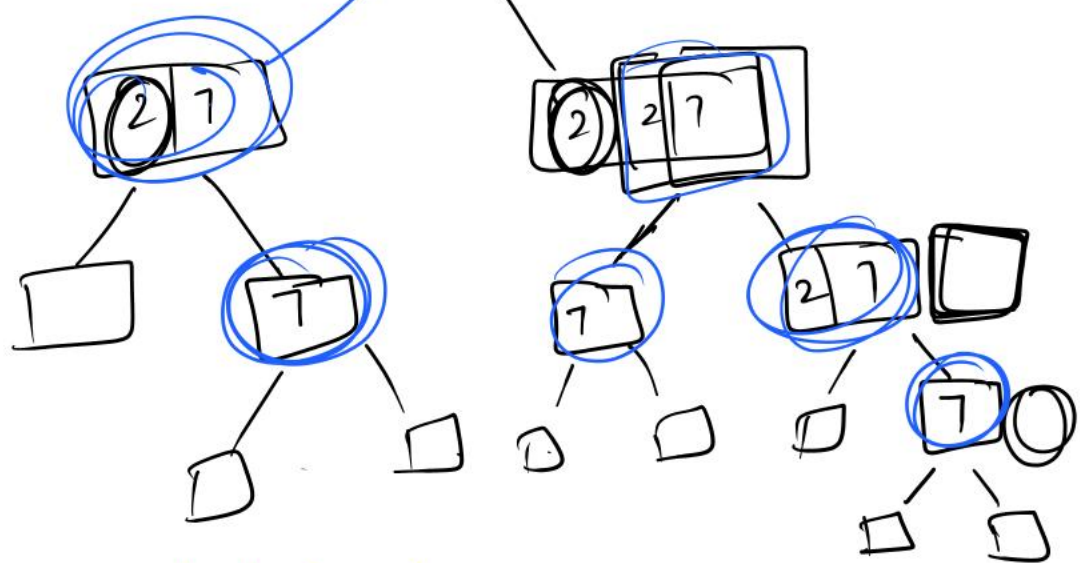


②

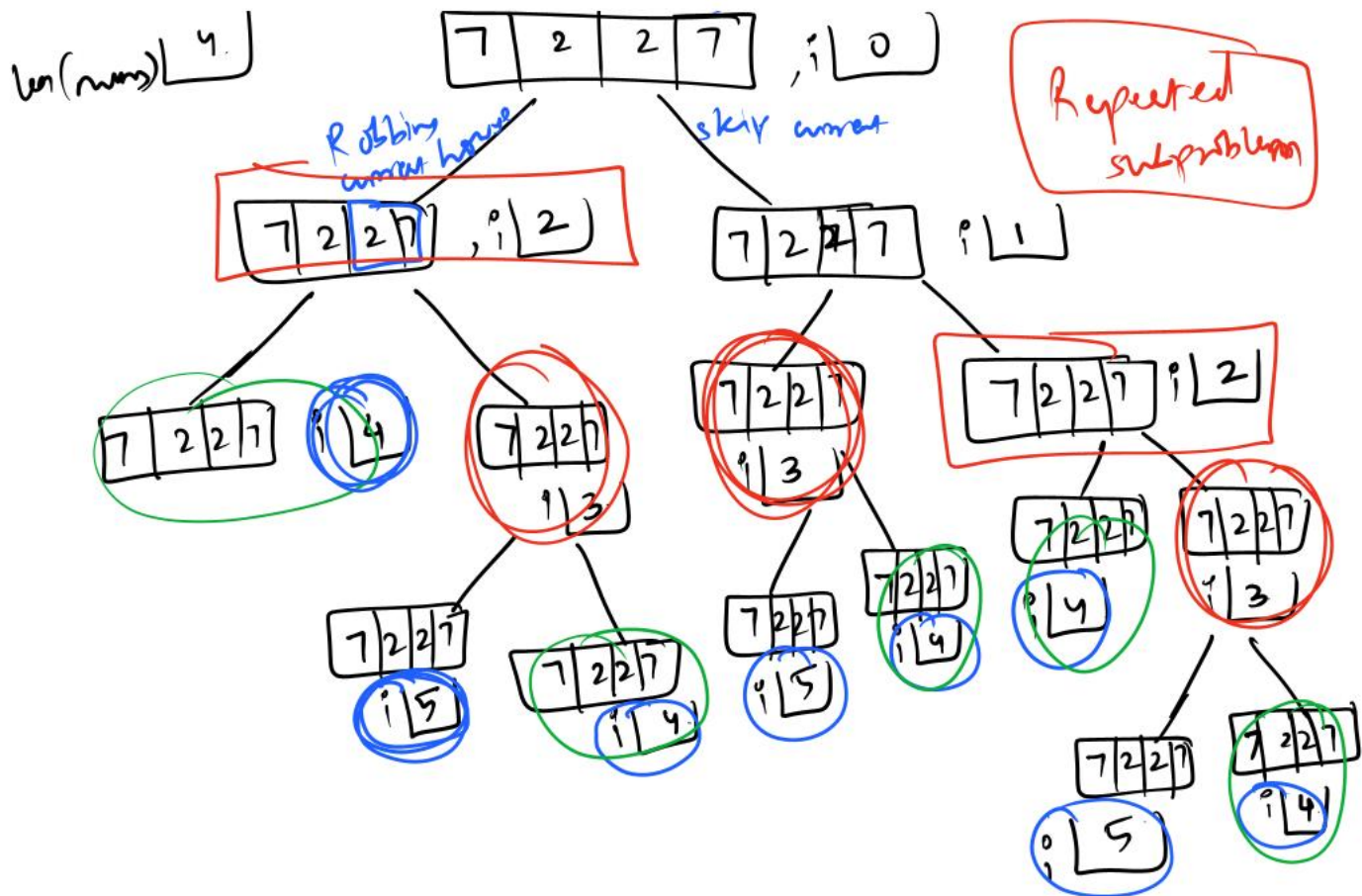
①



7	2	2	7
---	---	---	---



0 1 2 3



```

class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i):
            if i >= len(nums):
                return 0
            if dp[i] != -1:
                return dp[i]
            robCurrent = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            dp[i] = max(robCurrent, skipCurrent) # Storing the ans
            return dp[i]
        dp = [-1] * (len(nums) + 1)
        return helper(0)

```

memoization

call stack space

p.No : (198)

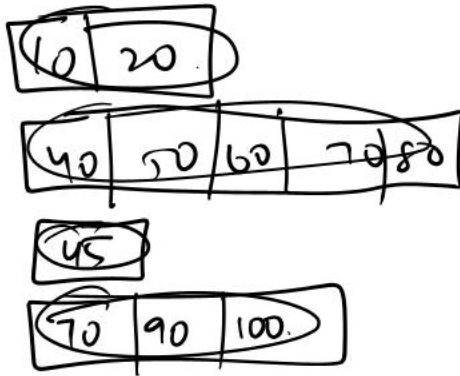
T.C $\rightarrow O(n)$

S.C $\rightarrow O(n)$

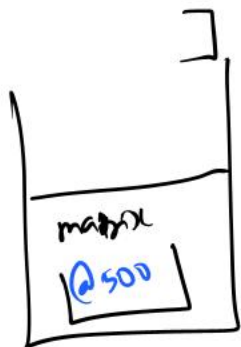
$\downarrow$ $n+n$ dp array's size

10 cases

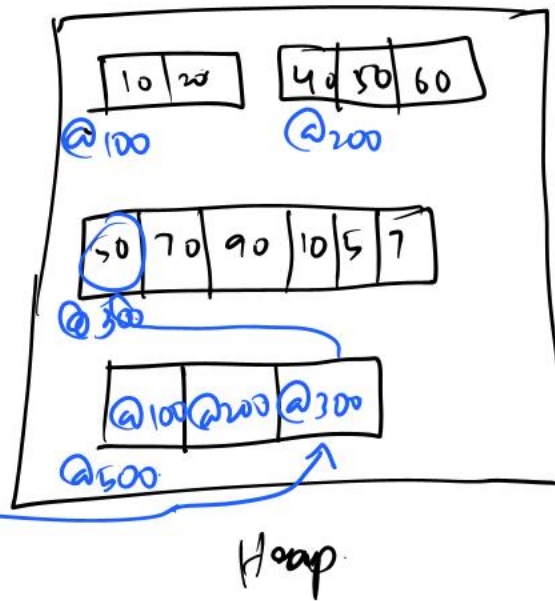
2D Matrix



matrix = [
[10, 20],
[40, 50, 60],
[50, 70, 70, 10, 5, 7]



→ matrix[2][0]
call stack



rows [3]

"10 20" → ["10", "20"]

1672. Richest Customer Wealth

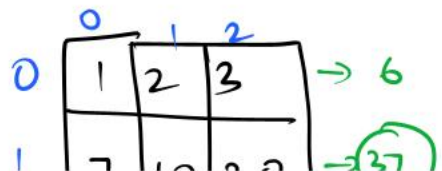
Solved ✓

Easy

Topics

Companies

Hint

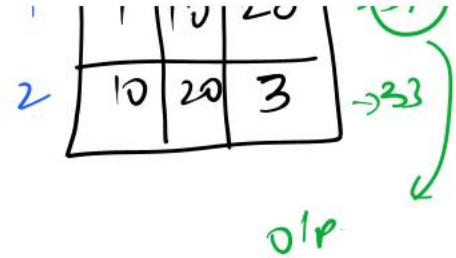


You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the wealth that the richest customer has.

A customer's wealth is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: `accounts = [[1,2,3], [3,2,1]]`
 Output: 6



```
class Solution:
    def maximumWealth(self, accounts: List[List[int]]) -> int:
        maximum = 0
        for i in range(len(accounts)) :
            currentWealth = 0
            for j in range(len(accounts[i])) :
                currentWealth += accounts[i][j]
            maximum = max(currentWealth, maximum)
        return maximum
```

$m \rightarrow \text{rows}$
 $n \rightarrow \text{cols}$

$\downarrow$
 $T.C \rightarrow O(m \times n)$

$\downarrow$
 $S.C \rightarrow O(1)$

$\downarrow$
 P.No: 1672

766. Toeplitz Matrix

Solved

Easy

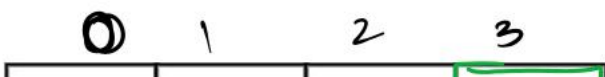
Topics

Companies

Hint

Given an $m \times n$ matrix, return true if the matrix is Toeplitz. Otherwise, return false.

A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.



$[0,0] \leftrightarrow [1,1]$

$[0,1] \leftrightarrow [1,2]$

0	1	2	3	4
1	5	1	2	3
2	9	5	1	2

$[0,1] \leftrightarrow [1,2]$
 $[0,2] \leftrightarrow [1,3]$
 $[1,0] \leftrightarrow [2,1]$
 $[1,1] \leftrightarrow [2,2]$
 $[1,2] \leftrightarrow [2,3]$
 $[i,j] \leftrightarrow [i+1,j+1]$

$i \rightarrow 0 \text{ to } m-1$
 $j \rightarrow 0 \text{ to } n-1$

	0	1	2	3
0	1	2	3	4
1	5	1	2	3
2	9	5	1	2

$i \rightarrow 1 \text{ to } m$

$j \rightarrow 1 \text{ to } n$

$[i][j] \leftrightarrow [i-1][j-1]$

```

class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        for i in range(len(matrix) - 1):
            for j in range(len(matrix[i]) - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False
        return True

```

P.No: 766

→ T.C
↓

$O(m \times n)$

```

class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        for i in range(1, len(matrix)):
            for j in range(1, len(matrix[i])):
                if matrix[i][j] != matrix[i - 1][j - 1]:
                    return False
        return True

```

→ S.C
↓
 $O(1)$

73. Set Matrix Zeroes

Medium

Topics

Companies

Hint

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:

1	1	1		1	0	1
1	0	1	→	0	0	0
1	1	1		1	0	1

	0	1	2	3
0	0	X	0	0
1	3	4	0	2
2	1	3	1	5

$\textcircled{3} \begin{array}{c} 2 \\ 1 \end{array} \left| \begin{array}{c} X \\ \cancel{2} \end{array} \right|$

$\textcircled{4} \begin{array}{c} 2 \quad 3 \quad 4 \\ 0 \end{array} \left| \begin{array}{c} X \\ \cancel{0} \end{array} \right|$

rows =	F	F	F
	0	1	2

cols =	F	F	F	F
	0	1	2	3

X	X	X
1	1	0

1	X	0
---	---	---

```
class Solution:
```

```
def setZeroes(self, matrix: List[List[int]]) -> None:
```

```
    m = len(matrix)
```

```
    n = len(matrix[0])
```

```
    rows = [False] * m
```

```
    cols = [False] * n
```

```
    # Find the row and col which are 0
```

```
    for i in range(m):
```

```
        for j in range(n):
```

```
            if matrix[i][j] == 0:
```

```
                rows[i] = True
```

→ p. no → 13

→ T.C → $O(m \times n)$

→ S.C → $O(m+n)$


```

        cols[j] = True
    # Set the matrix with 0
    for i in range(m):
        for j in range(n):
            if rows[i] or cols[j]:
                matrix[i][j] = 0

```

Handwritten notes: $m \times n$ matrix, m rows, n cols.

Handwritten: rows + cols

999. Available Captures for Rook

Solved

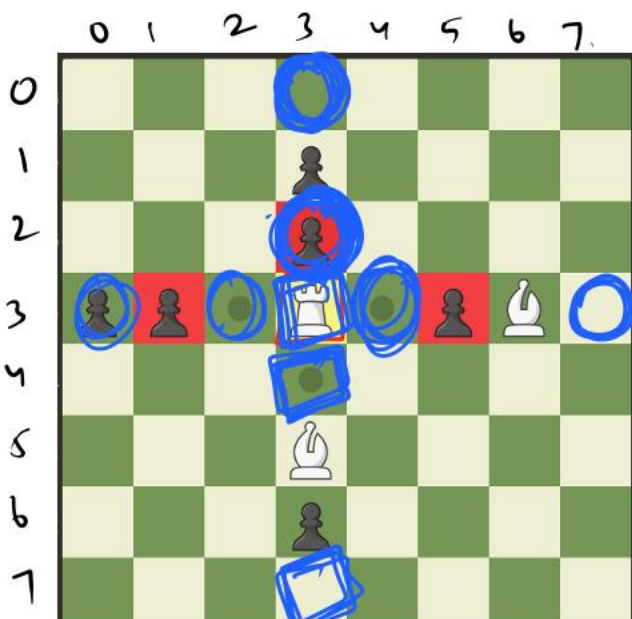
Easy Topics Companies

You are given an 8×8 matrix representing a chessboard. There is exactly one white rook represented by 'R', some number of white bishops 'B', and some number of black pawns 'p'. Empty squares are represented by '.'.

A rook can move any number of squares horizontally or vertically (up, down, left, right) until it reaches another piece or the edge of the board. A rook is **attacking** a pawn if it can move to the pawn's square in one move.

Note: A rook cannot move through other pieces, such as bishops or pawns. This means a rook cannot attack a pawn if there is another piece blocking the path.

Return the number of pawns the white rook is **attacking**.



Handwritten: $R \rightarrow [3,3]$ $i[3]$ $j[3]$

Direction	common	start	end
up	j	$i-1$	0
down	j	$i+1$	7
right	i	$j+1$	7
left	i	$j-1$	0


```

class Solution:
    def numRookCaptures(self, board: List[List[str]]) -> int:
        pawns = 0
        # Find rook position
        for i in range(8):
            for j in range(8):
                if board[i][j] == 'R':
                    # Explore all 4 directions
                    # Up -> j is common, i - 1 to 0
                    for k in range(i - 1, -1, -1):
                        # Bishop
                        if board[k][j] == 'B':
                            break
                        elif board[k][j] == 'p':
                            pawns += 1
                            break
                    # Down -> j is common, i + 1 to 7
                    for k in range(i + 1, 8):
                        # Bishop
                        if board[k][j] == 'B':
                            break
                        elif board[k][j] == 'p':
                            pawns += 1
                            break

```

```

# Right -> i is common, j + 1 to 7
for k in range(j + 1, 8):
    # Bishop
    if board[i][k] == 'B':
        break
    elif board[i][k] == 'p':
        pawns += 1
        break

# Left -> i is common, j - 1 to 0
for k in range(j - 1, -1, -1):
    # Bishop
    if board[i][k] == 'B':
        break
    elif board[i][k] == 'p':
        pawns += 1
        break

return pawns

```

↓
P.NO : (999)

↓
T.C → O(1)

↓
S.C → O(1)

64. Minimum Path Sum

Solved ✓

Medium

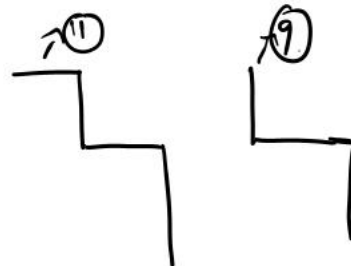
Topics

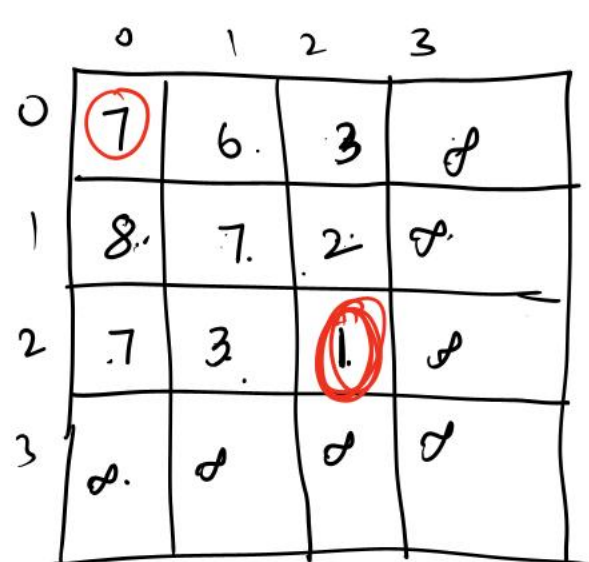
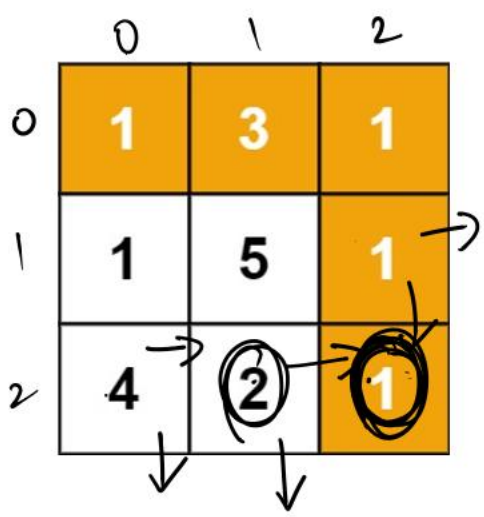
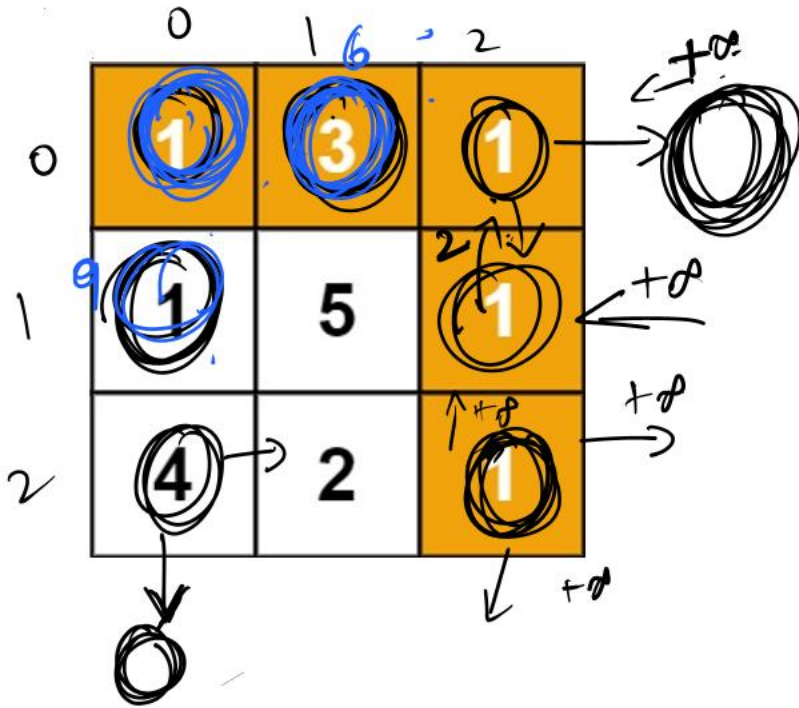
Companies

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

	0	1	2
0	1	3	1
1	1	5	1
2	4	2	1





```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        dp = [
```

→ p.no: 64

```

dp = []
for i in range(m + 1) :
    nums = []
    for j in range(n + 1):
        nums.append(float('inf'))
    dp.append(nums)
for i in range(m - 1, -1, -1) :
    for j in range(n - 1, -1, -1) :
        if i == m - 1 and j == n - 1 :
            dp[i][j] = grid[i][j]
            continue
        dp[i][j] = grid[i][j] + min(dp[i][j + 1], dp[i + 1][j])
return dp[0][0]

```

→ T.C → $O(m \times n)$

→ S.C → $O(m \times n)$

→ ITERATIVE
DP

1572. Matrix Diagonal Sum

Easy

Topics

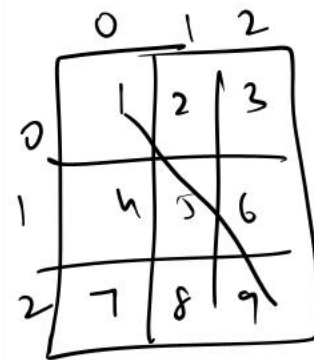
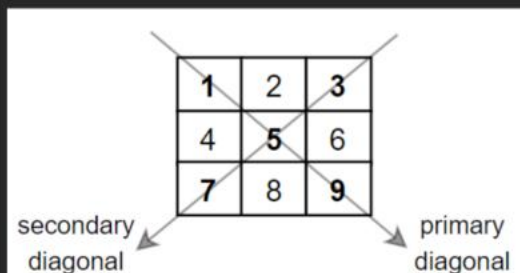
Companies

Hint

Given a square matrix `mat`, return the sum of the matrix diagonals.

Only include the sum of all the elements on the primary diagonal and all the elements on the secondary diagonal that are not part of the primary diagonal.

Example 1:



primary diagonal



$[0,0], [1,1], [2,2]$

$[0,2], [1,1], [2,0]$

$i + j == n - 1$

→ Secondary diagonal

$i == j$ → Primary diagonal

class Solution:

def diagonalSum(self, mat: List[List[int]]) -> int:

ans = 0

n = len(mat)

for i in range(n) :

for j in range(n) :

if $i == j$ or $i + j == n - 1$:

ans += mat[i][j]

→ P.No 1572

→ T.C → $O(n^2)$

→ S.C → $O(1)$

return ans

$n=3$

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

$$[i][j] + [i][n-1-j]$$

$i \rightarrow 0$

$i \rightarrow 1$

$i \rightarrow 2$

$$[0][0] + [0][2]$$

$$[1][1] + [1][1]$$

$$[2][2] + [2][0]$$

$$[n//2] [n//2]$$

```
class Solution:
```

```
def diagonalSum(self, mat: List[List[int]]) -> int:
```

```
    ans = 0
```

```
    n = len(mat)
```

```
    for i in range(n):
```

```
        ans += mat[i][i]
```

```
        ans += mat[i][n - 1 - i]
```

```
    if n % 2 == 1:
```

```
        ans -= mat[n // 2][n // 2]
```

```
    return ans
```

$\rightarrow$ p.no:

1512

$\rightarrow$ T.C -

$\downarrow$

$O(n)$

S.C -

$O(1)$